DIGITAL DESIGN PROJECT

Secure UART Communication System

UART Interface with Hardware AES-128 Encryption and CRC-16/XMODEM Error Detection

Prepared by: **Mina Moawad**, **Muhammed Afifi** , **Hoda Ayman**

Program: **Digital Design Phase**

Supervisor: **Eng/Muhammed Salah**

Institution: **NTI**

Date: 7/9/2026

Contents

- Document Control 4
- Abstract: 4

Introduction 5

1. Project overview: 5

1.1 project objectives: 5

1.2 Functional Summary: 5

System Requirements: 6

2.1 Functional Requirements: 6

2.2 Assumed/ Configurable Parameters: 6

Literature Review 7

3.1 AES Integration with UART: 7

3.2 Error Detection over Serial Links: 7

3.3 Positioning of This Project: 8

System Architecture 8

4.1 Transmit Path: 8

4.2 Block-Level Responsibilities: 9

4.3 Positioning of This Project: 10

4.4 Byte Serialization of AES Result: 10

UART Communication Module 11

5.1 UART Transmitter Operation: 11

5.2 UART Receiver Operation: 11

5.3 UART Interface: 12

5.4 UART Testbench: 13

AES-128 Encryption Module 14

6.1 AES-128 Encryption Structure: 15

6.2 AES State Transformation: 16

6.3 AES Key Expansion: 17

6.4 AES Control Sequence: 17

6.5 AES Interface 18

6.6 AES Testbench: 18

CRC-16/XMODEM Module 20

7.1 CRC-16/XMODEM Parameters: 20

7.2 CRC Calculation Concept: 21

7.3 CRC Interface: 21

7.4 CRC Processing Architecture: 21

7.5 CRC Testbench: 22
End-to-End Data Path and Integration 24
8.1 Recommended Transmission Sequence: 24
8.2 Integrated State Machine: 24
Control and Handshake Strategy 25
9.1 Busy Protection: 25
9.2 Done Pulse Handling: 25
AES Integration with CRC 26
System Integration 28
Timing and Synthesizability Considerations 31
10.1 AES Timing: 31
10.2 CRC Timing: 31
10.3 UART Timing: 31
10.4 Reset Strategy: 31
Design Decisions and Engineering Rationale 32
11.1 Security and Integrity: 32
11.2 CRC After Encryption: 32
Limitations and Future Improvements 33
Final Specifications 33
Conclusion 34
References 34

❖ Document Control

Document Item	Value
Project Scope	UART communication with hardware AES-128 encryption and CRC-16/XMODEM integrity checking
Implementation	Verilog HDL / RTL
Cryptographic Algorithm	AES-128 encryption
Integrity Algorithm	CRC-16/XMODEM
Communication	UART
System Status	Individual modules implemented and integrated into a complete RTL system
Document Type	Technical Design, Architecture, Operation and Verification Report

❖ Abstract:

This report documents a complete hardware communication and security data path implemented in Verilog/SystemVerilog HDL. The design combines a UART communication interface, an AES-128 encryption engine, and a CRC-16/XMODEM error-detection engine into an integrated RTL system. Application data is encrypted with AES-128, a CRC-16/XMODEM checksum is calculated over the resulting ciphertext, and the protected frame is transmitted serially through UART. The report covers the project's motivation and requirements, a review of related work, the system architecture, the detailed design of each RTL module (UART, AES-128, CRC-16/XMODEM), the end-to-end integration and handshake strategy, the verification plan (including the AES-128 known-answer vector and the CRC-16/XMODEM standard check value).

Introduction

1. Project overview:

This project implements a complete hardware communication and security datapath in Verilog/SystemVerilog HDL. The design combines a UART communication interface, an AES-128 encryption engine, and a CRC-16/XMODEM error-detection engine into an integrated RTL system. The objective is to demonstrate how communication, confidentiality, and data-integrity functions can be implemented and coordinated at the hardware level.

The system accepts application data, encrypts the data using AES-128, calculates a CRC-16/XMODEM checksum over the defined protected data field, and transmits the resulting frame through UART. On the receiving side, the corresponding data can be recovered and checked using the matching protocol. The integrated architecture therefore provides three distinct functions: reliable serial transport, confidentiality, and error detection.

1.1 project objectives:

- Implement a synthesizable UART transmitter/receiver subsystem in Verilog/SystemVerilog HDL.
- Implement AES-128 encryption as a hardware RTL block.
- Implement CRC-16/XMODEM using the standard CRC parameters and verify its calculation.
- Integrate the three functions into a single end-to-end communication datapath.
- Define clear start/busy/done handshaking between processing stages.
- Verify individual modules and the integrated system using simulation-based testbenches.
- Produce an RTL architecture that can be synthesized and used as a foundation for FPGA or ASIC implementation.

1.2 Functional Summary:

Block	Primary Function	Main Data	Completion / Status
UART	Serial communication	Parallel byte ↔ serial bitstream	busy / done and RX indication
AES-128	Confidentiality	128-bit plaintext + 128-bit key → 128-bit ciphertext	busy / done
CRC-16/XMODEM	Integrity / error detection	Input data → 16-bit CRC	busy / done
Top-Level Integration	Sequencing and framing	End-to-end protected payload	System control / status

System Requirements:

2.1 Functional Requirements:

- The UART shall serialize and deserialize data according to the selected baud-rate configuration.
- The UART transmitter shall append the required start and stop bits to each transmitted byte.
- The UART receiver shall detect the incoming frame and reconstruct the received byte.
- The AES block shall perform AES-128 encryption on a 128-bit plaintext using a 128-bit master key.
- The CRC block shall calculate a 16-bit CRC using the CRC-16/XMODEM algorithm.
- The integrated controller shall prevent downstream stages from consuming incomplete or invalid data.
- The integrated system shall indicate completion and/or availability of results through defined status signals.

2.2 Assumed/ Configurable Parameters:

Parameter	Description	Project Treatment
UART clock	System clock used by the UART timing logic	Defined by the implementation/testbench
Baud rate	Serial bit rate	Configurable/selected according to the target system
UART frame	Start + data + stop format	Implemented by the UART subsystem
AES key size	Master key length	128 bits
AES block size	Data block length	128 bits
CRC width	Checksum width	16 bits
CRC polynomial	Generator polynomial	x^16 + x^12 + x^5 + 1 (0x1021)
CRC initial value	Initial register state	0x0000
CRC input reflection	RefIn	False

Parameter	Description	Project Treatment
CRC output reflection	RefOut	False
CRC final XOR	XorOut	0x0000

Literature Review

3.1 AES Integration with UART:

Basu, Kole and Rahaman (2012) describe an early implementation of the AES algorithm inside a UART module to secure serial data transfer, showing that block-cipher encryption can be embedded directly in the serial data path rather than performed in software on a host processor. Katkade and Phade (2016) extend this idea, applying AES specifically to protect data confidentiality in serial communication links and reporting on the practical trade-offs of adding a cipher to a low-resource serial interface.

Kala, Panda and Tanwar (2023) target a high-throughput, low-power AES-128 cipher implementation on FPGA, highlighting the area/throughput trade-off between fully unrolled (pipelined) AES round datapaths and iterative, single-round-reuse datapaths. the latter being the approach adopted for the AES-128 core documented in Section 6, where the same physical round data path is reused across all ten rounds under FSM control.

It presents a system that encrypts a 128-bit plaintext block with a fixed AES key for secure UART transfer, reinforcing the practice of pairing AES confidentiality with an explicit, well-documented frame convention the approach followed here with the ciphertext-plus-CRC protected payload.

3.2 Error Detection over Serial Links:

Because UART itself provides no automatic-repeat-request or acknowledgment mechanism, corrupted or truncated frames can otherwise be silently accepted by a receiver. Prior work combining UART with CRC-based error detection demonstrates that a cyclic redundancy check substantially improves reliability compared with parity alone, particularly for multi-byte frames where several bits may be corrupted by burst noise. This project uses the CRC-16/XMODEM parameterization specifically polynomial 0x1021 with a non-reflected, zero-

initialized register (init 0x0000, RefIn/RefOut false, XorOut 0x0000) — which differs from the closely related CRC-16/CCITT-FALSE variant only in its initial value.

3.3 Positioning of This Project:

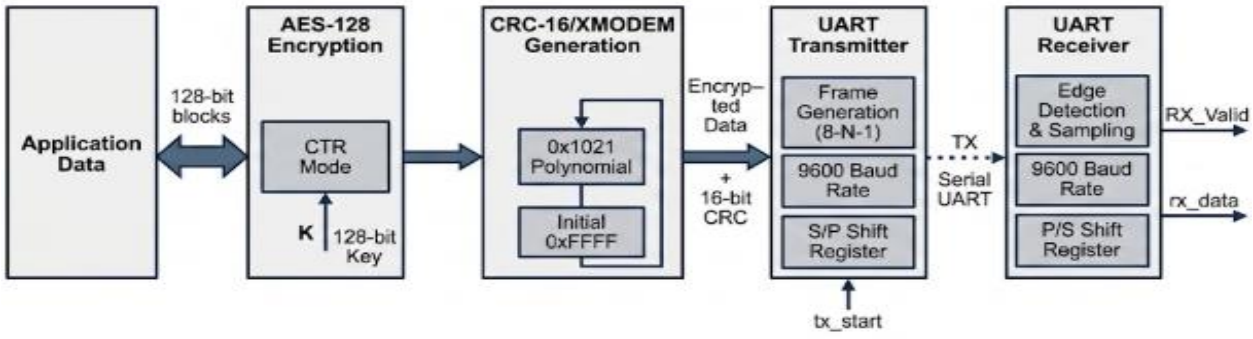
Relative to the works above, this project's contribution is the coordination of three independently verifiable RTL blocks — an iterative AES-128 core, a byte-oriented CRC-16/XMODEM engine, and a UART transceiver — through an explicit start/busy/done handshake protocol and a top-level FSM, together with a documented verification plan built on standard, published test vectors for both AES-128 and CRC-16/XMODEM.

System Architecture

The integrated system is organized as a sequence of hardware processing stages. The exact top-level framing can be adapted to the selected application, but the logical relationship between the implemented functions is fixed.

4.1 Transmit Path:

Application Data → AES-128 Encryption → CRC-16/XMODEM Generation → UART Transmitter → UART RX



4.2 Block-Level Responsibilities:

Stage	Input	Processing	Output
Input / Application	Raw application data	Load and validate payload	128-bit data block / byte stream
AES-128	128-bit plaintext + 128-bit key	Key expansion + 10 encryption rounds	128-bit ciphertext
CRC-16/XMODEM	Defined protected data field	Polynomial division / CRC update	16-bit CRC
Framing	Ciphertext + CRC	Construct transmission sequence	UART byte stream
UART TX	Parallel byte data[7:0] + tx_en request	Frame assembly {stop,data[7:0], start} (LSB-first) and tick-driven bit-serial shift-out via the IDLE/ON FSM	Serial TX bitstream (tx), plus busy/done status
UART RX	Serial RX bitstream	Falling-edge detection → 1.5-bit then 1-bit mid-bit sampling via the down-counter → SIPO shift-in → stop-bit validation	Reassembled byte data[7:0], plus busy/done/err status
AES-128 Decryption	128-bit Ciphertext + 128-bit key	Key expansion + 10 Decryption rounds	128-bit plaintext
CRC Check	Data with CRC bits	Polynomial division/check CRC	Valid or corrupted frame

4.3 Positioning of This Project:

A practical protected frame is organized as ciphertext followed by CRC. The exact byte ordering is fixed between transmitter and receiver and documented as part of the interface specification.

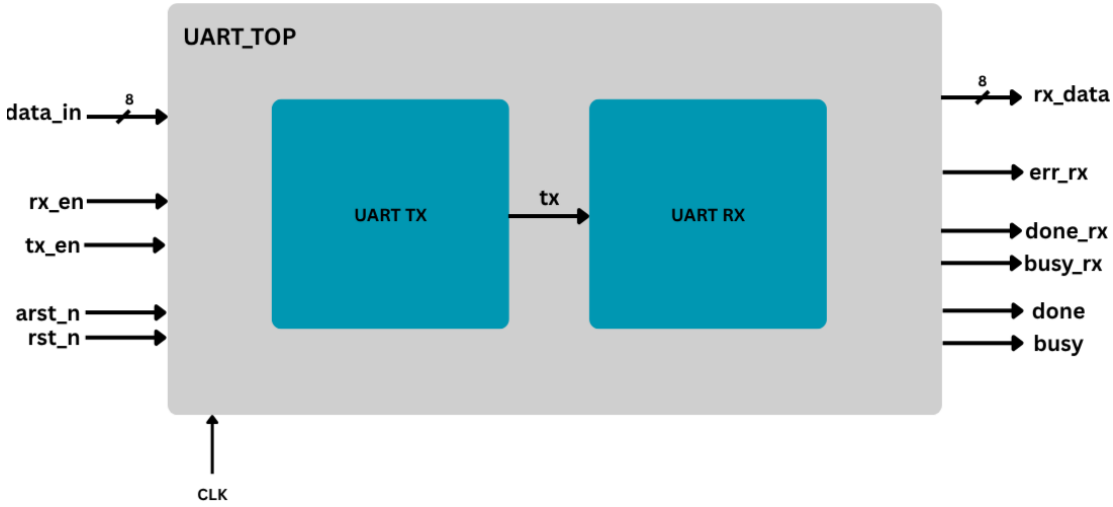
The CRC must be calculated over exactly the same byte sequence that the receiver uses for verification. The specification should explicitly define whether the CRC covers plaintext, ciphertext, header fields, or another protected field.

Field	Size	Purpose
Ciphertext	16 bytes	AES-128 encrypted application block
CRC	2 bytes	CRC-16/XMODEM integrity check
Total protected payload	18 bytes	Data transmitted after framing into UART bytes

4.4 Byte Serialization of AES Result:

The 128-bit AES result is converted into 16 bytes for UART transmission. The implementation must define a consistent byte ordering. The most-significant byte is transmitted first, the sequence is Ciphertext [127:120] → Byte 0, Ciphertext [119:112] → Byte 1, Ciphertext [7:0] → Byte 15, followed by the two CRC bytes in the documented order (high byte then low byte).

UART Communication Module



UART (Universal Asynchronous Receiver/Transmitter) provides the serial communication interface of the system. Because UART is asynchronous at the physical protocol level, the transmitter generates the serial waveform from the local clock while the receiver reconstructs data from the incoming RX signal using its configured bit timing. The system operates on a standard 8-N-1 frame format (1 start bit, 8 data bits LSB first, no parity bit, 1 stop bit) at a default baud rate of 9600 baud using a 50 MHz system clock.

5.1 UART Transmitter Operation:

- **Idle State:** Wait in the idle state (IDLE) with the tx output line held high (logic 1).
- **Load Frame:** Construct 10 – bit frame {1, data, 0} with start (0), 8 data bits, and stop (1) bits.
- **Transmit:** Transition to ON state, assert busy, and output bits sequentially indexed by baud rate tick pulses.
- **Complete:** clear busy, pulse done high, reset index, and return to IDLE after transmitting all 10 bits.

The busy signal is important at the system level: it prevents a new byte from overwriting the currently transmitted byte. The done signal can be used by the integrated controller to advance to the next byte.

5.2 UART Receiver Operation:

- Start Detection: monitor rx in IDLE; assert falling edge on a high to low transition.
- Mid-Bit Sampling: Trigger the baud counter to wait 1.5-bit period to sample the stat bit midpoint, then reload 1-bit period for subsequent Data bits.

- Data Shift: Assert en_shift signal on each finish tick to shift incoming rx bits LSB-first into the SIPO register.
- Stop Bit & Validation: Sample the stop bit after 8 data bits; assert done if rx is high or enter ERR state and assert err if low.

5.3 UART Interface:

The UART interface is byte-oriented, while AES operates on 128-bit blocks and CRC produces a 16-bit result. The top-level controller is therefore responsible for converting the block-level results into an ordered byte stream.

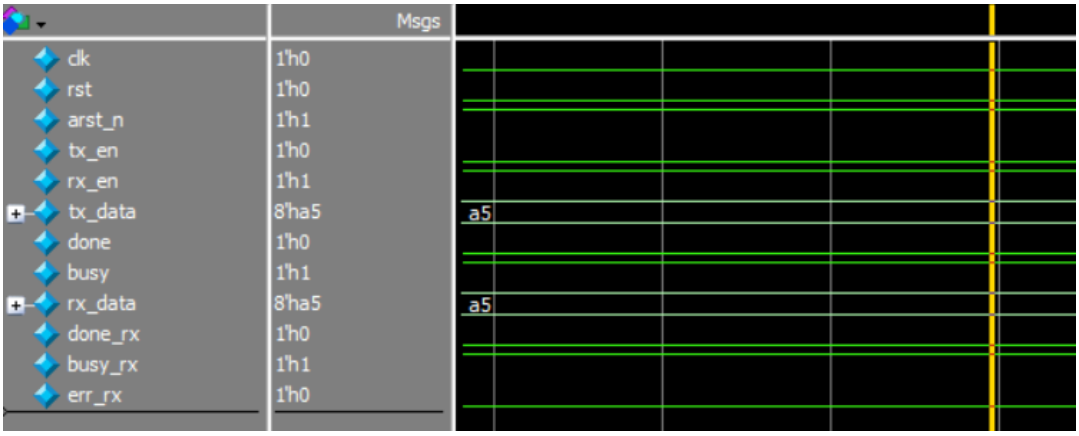
Signal	Direction	Description
clk	Input	System clock signal (50 MHz)
rst	Input	synchronous active-high reset signal
arst_n	Input	Asynchronous active-low reset signal
tx_en	Input	Enable signal to initiate transmission of loaded byte
data (TX)	Input	8-bit parallel byte to transmit
tx	Output	Serial UART transmit line
busy (TX)	Output	High while a byte transmission is in progress
done (TX)	Output	Pulse high indicating completion of byte transmission
rx_en	Input	Enable signal for receiver module
rx	Input	Serial UART receive line
data (RX)	Output	8-bit recovered received data byte
done (RX)	Output	Pulse high indicating valid byte successfully received
busy (RX)	Output	High while frame reception is in progress

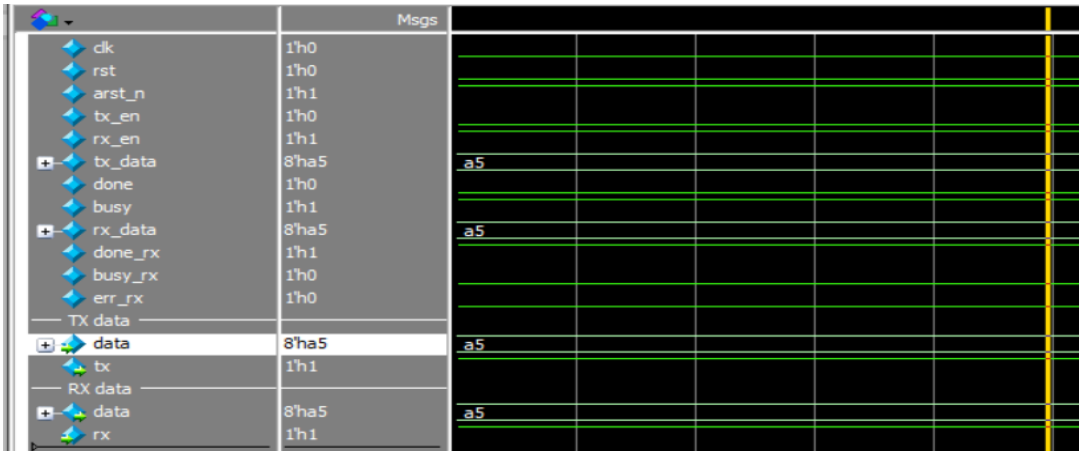
Signal	Direction	Description
err	Output	indicates a framing error (invalid stop bit)

5.4 UART Testbench:

The provided test verifies the full loopback functionality of the top-level UART module. This is done using a 50 MHz clock setup.

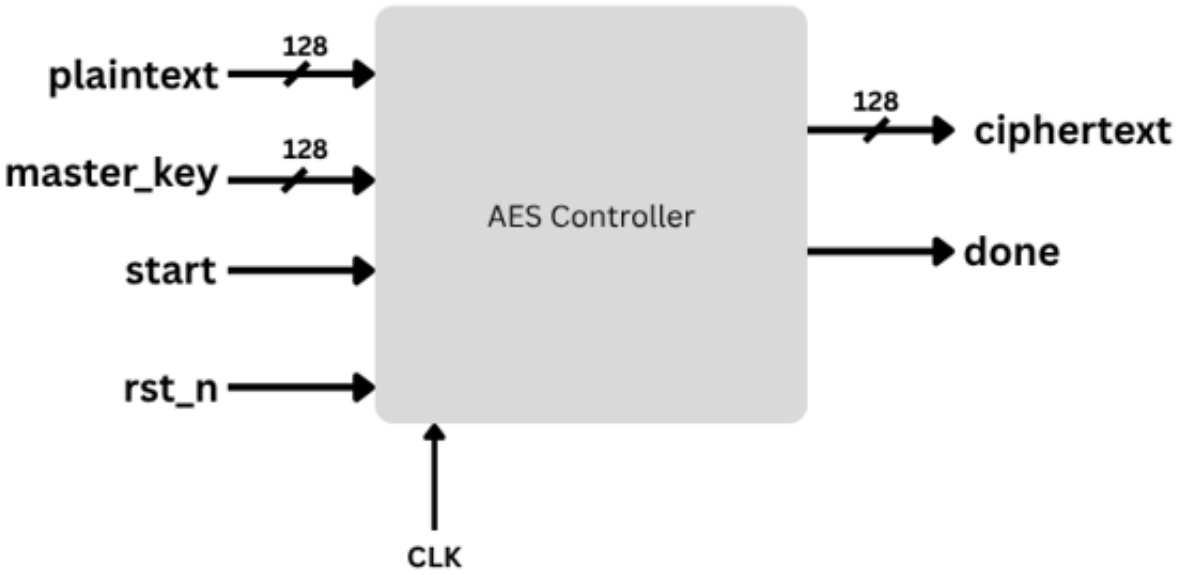
Test	Input Data	Expected received data
TEST 1	8'hA5	8'hA5
TEST 2	8'h55	8'h55
TEST 3	8'hFF	8'hFF





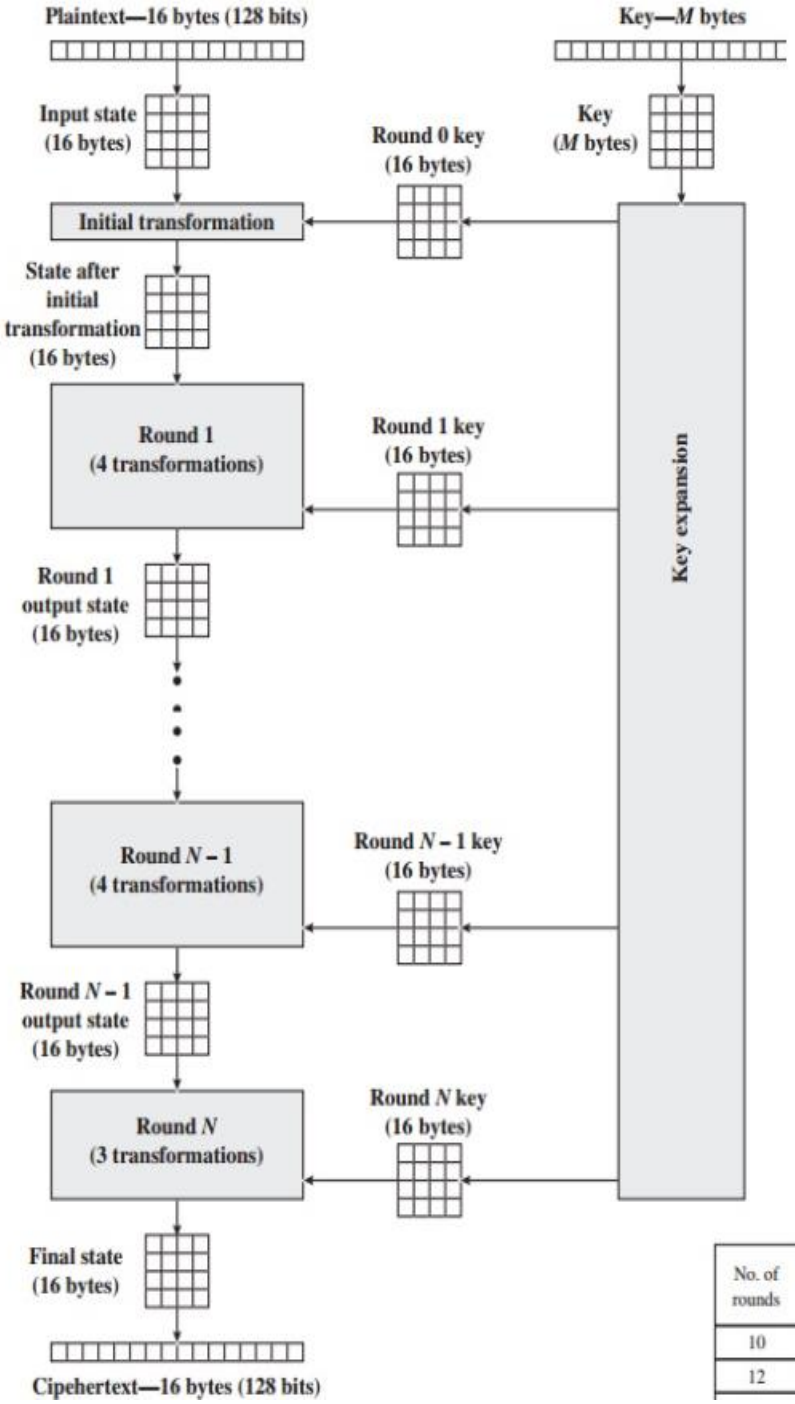
AES-128 Encryption Module





AES (Advanced Encryption Standard) is used to provide confidentiality. The implemented AES engine operates on a 128-bit plaintext block and a 128-bit master key, producing a 128-bit cipher text block. AES-128 uses 10 encryption rounds in addition to the initial AddRoundKey operation.

6.1 AES-128 Encryption Structure:

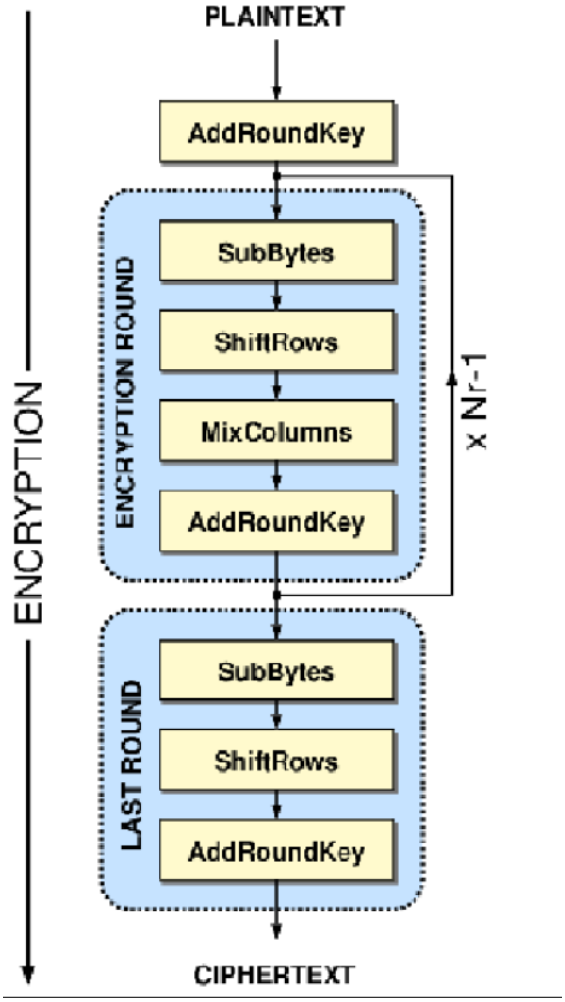


No. of rounds	Key Length (bytes)
10	16
12	24

Operation	Purpose
Initial AddRoundKey	XOR the plaintext state with the initial round key.
Rounds 1–9	SubBytes → ShiftRows → MixColumns → AddRoundKey.
Round 10	SubBytes → ShiftRows → AddRoundKey; MixColumns is omitted.

6.2 AES State Transformation:

- SubBytes: substitutes each byte using the AES S-box.
- ShiftRows: cyclically shifts rows of the 4×4-byte state.
- MixColumns: applies the AES matrix transformation to each column.
- AddRoundKey: XORs the state with the current 128-bit round key.



6.3 AES Key Expansion:

The AES-128 key schedule expands the 128-bit master key into eleven 128-bit round keys, K0 through K10. K0 is used by the initial AddRoundKey operation, while K1 through K10 correspond to encryption rounds 1 through 10.

```
AES-128 key schedule


Master Key -> K0
  |
  K1 -> Round 1
  K2 -> Round 2
  ...
  K9 -> Round 9
  K10 -> Round 10
```

For encryption, the round-key order is therefore K0 at the initial transformation followed by K1, K2, …, K10. For a corresponding AES decryption implementation (noted as future work in Section 11), the round keys are consumed in reverse order, subject to the exact inverse-round architecture chosen.

6.4 AES Control Sequence:

State	Action
IDLE	Wait for a new encryption request.
LOAD	Capture plaintext and master key.
KEY_EXPANSION	Generate/store the AES round keys.
INITIAL_ADD_KEY	Perform plaintext XOR K0.
ROUND_1 – ROUND_9	Execute the standard full AES round.
ROUND_10	Execute the final AES round without MixColumns.
DONE	Register the ciphertext and signal completion.

A clocked FSM-based implementation is suitable for this architecture because it makes the sequencing of the ten AES rounds explicit and permits the same physical round datapath to be reused across rounds, minimizing hardware area compared with a fully unrolled implementation

6.5 AES Interface

Signal	Description
clk	System clock
rst_n	Active-low reset, according to the implemented interface
start	Starts an AES encryption operation
plaintext[127:0]	128-bit input data block
master_key[127:0]	128-bit AES-128 master key
ciphertext[127:0]	128-bit encrypted output block
busy	Indicates that AES is processing
done	Indicates that ciphertext is valid

6.6 AES Testbench:

The provided testbench (tb_AES_FSM) verifies the functional correctness and state-machine control flow of the core AES-128 Encryption engine (AES_FSM). It applies clock generation, synchronous reset sequencing, input data latching, and execution pulse triggers while dynamically monitoring simulation signals and reporting final ciphertext results.

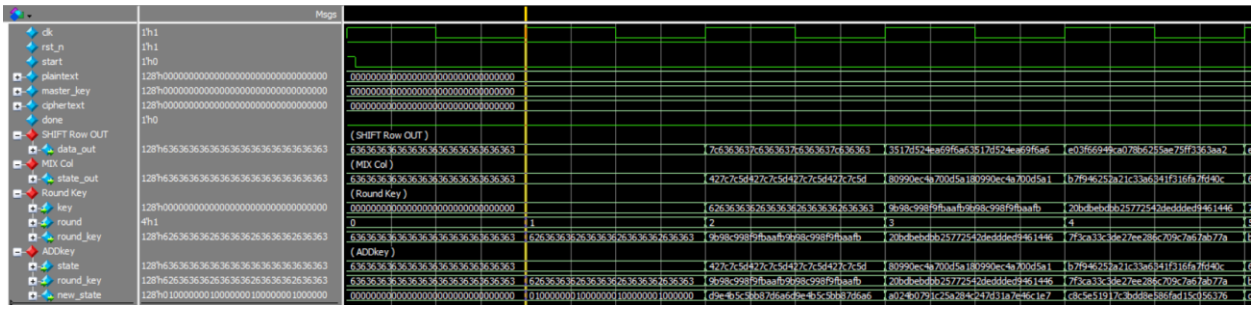
Reset Phase: All control inputs (start, plaintext, master_key) are initialized to zero, and active-low reset (rst_n = 0) is held low for two clock cycles before release (rst_n = 1).

Data Latching Phase: 128-bit test vectors for plaintext and master_key are assigned across a clock edge using non-blocking assignments (<=) with a #1 delta delay to prevent setup time violations.

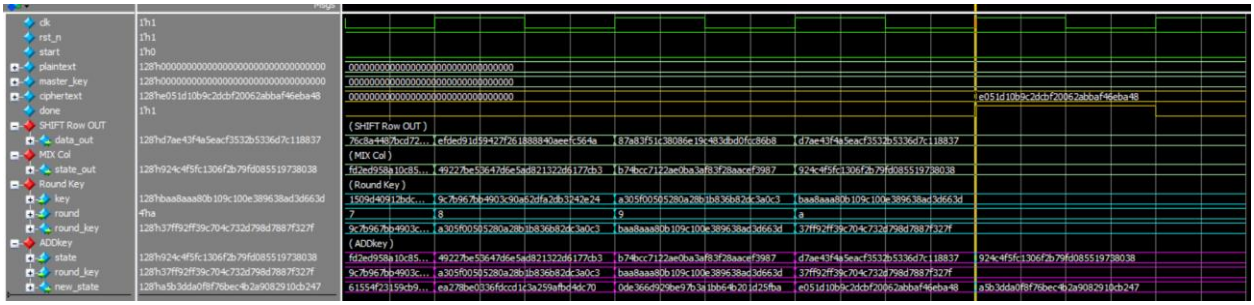
Execution Pulse: The start signal is pulsed high for exactly one clock cycle to transition the FSM from AES_IDLE into AES_ADD_ROUND_KEY_0, initiating the key expansion and 10-round substitution -permutation operations

Completion Synchronization: The testbench halts procedural execution using wait(done); until the FSM completes all 10 rounds and asserts the single-cycle done completion signal.

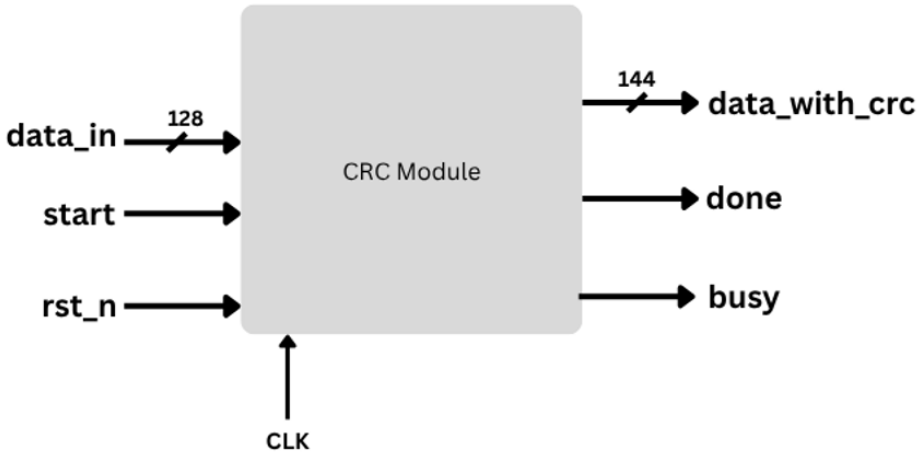
Signal Parameter	Bit Width	Expected value
Plaintext Input	128 bits	128'h00000000_00000000_00000000_00000000
Master Key Input	128 bits	128'h00000000_00000000_00000000_00000000
Expected Cipher text Output	128 bits	128'h66e94bd4_ef8a2c3b_884cfa59_ca342b2e







CRC-16/XMODEM Module



The CRC module implements a **CRC-16/XMODEM** error-detection algorithm in synthesizable Verilog HDL. Its purpose within the complete UART/AES/CRC system is to provide **error detection for transmitted data**. While AES-128 provides confidentiality, CRC provides a mechanism for detecting accidental corruption of the transmitted information. The implemented module accepts a **128-bit input data word** and generates a **16-bit CRC value**, resulting in a **144-bit output** containing the original data followed by its calculated CRC. The module is parameterized to allow the data width and CRC width to be modified.

7.1 CRC-16/XMODEM Parameters:

Parameter	Value
CRC width	16 bits
Polynomial	0x1021
Polynomial expression	x^16 + x^12 + x^5 + 1
Initial value	0x0000
RefIn	False
RefOut	False
XorOut	0x0000
Check value	0x31C3 for the standard ASCII string “123456789”

7.2 CRC Calculation Concept:

The CRC engine treats the input as a sequence of bits/bytes and updates a 16-bit register using polynomial division. For a non-reflected CRC-16/XMODEM implementation, each input byte is processed from the most significant bit toward the least significant bit.

7.3 CRC Interface:

Signal	Description
clk	System clock
rst_n	Reset input according to the implementation
start	Requests a CRC calculation
data_in	Input data block
busy	High while the CRC calculation is active
done	High/pulsed when the CRC result is available
data_with_crc	Calculated CRC-16/XMODEM value

7.4 CRC Processing Architecture:

The CRC generator employs a **sequential bit-by-bit architecture** to calculate the CRC-16/XMODEM checksum over a 128-bit input data block.

The input data is first captured in an interna-l register, while a 16-bit CRC register stores the intermediate checksum and a counter tracks the number of processed bits. The CRC calculation is initialized to **0x0000** and performed **MSB-first**, starting from data_in[127] and progressing to data_in[0]. One input bit is processed per clock cycle. For each bit, the feedback value is obtained by XORing the current MSB of the CRC register with the input bit. The CRC register is then shifted left, and the generator polynomial **0x1021** is XORed with the result when the feedback value is logic 1.

The operation is implemented within a **single clocked process without a dedicated FSM**. When the module is idle (busy = 0), the input is captured and the CRC calculation is initialized. While busy = 1, the module processes one bit per clock cycle until all 128 bits have been evaluated, requiring **128 processing cycles**. Upon completion, the calculated 16-bit CRC is appended to the original 128-bit data to form a **144-bit output frame**, with the data occupying the upper 128 bits

and the CRC occupying the lower 16 bits. The done signal indicates completion and the availability of the valid output, while busy indicates that the CRC engine is actively processing a data block.

7.5 CRC Testbench:

The provided testbench verifies the RTL using three independent 128-bit input vectors including the 0x31C3 check for the standard ASCII string “123456789”.

The testbench first applies reset:

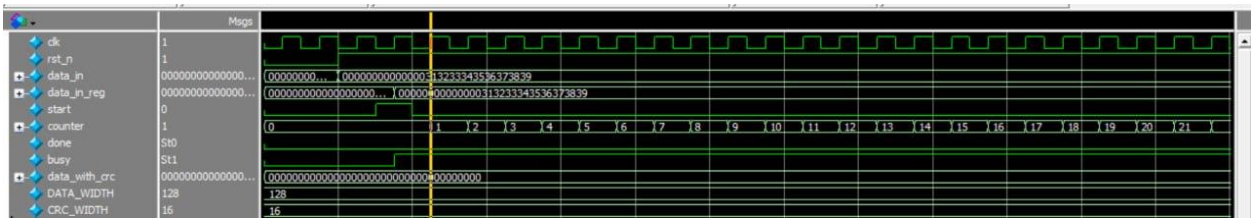
After reset is released, each test waits until the CRC module is idle before applying a new input. This is important because the module automatically begins a new calculation when busy is low.

The testbench then waits for the done event:

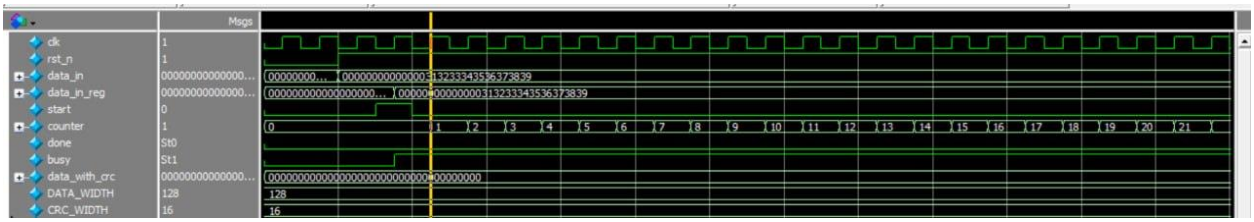
Once done is asserted, the testbench compares: data_with_crc[15:0] against the expected CRC value.

Test	Input Data	Expected CRC
TEST 1	128'h00000000000000313233343536373839	16'h31C3
TEST 2	128’h00112233445566778899AABBCCDDEEFF	16'h1248
TEST 3	128’h11223344556677889AABBCCDDEEAA55	16'h8169

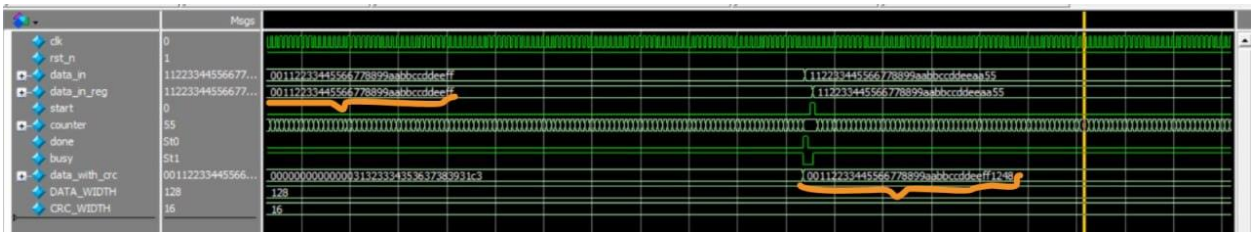
- First Test Case: The Counter starts after the *start* signal is high and the *data_in_reg* captured the data in *data_in* register:



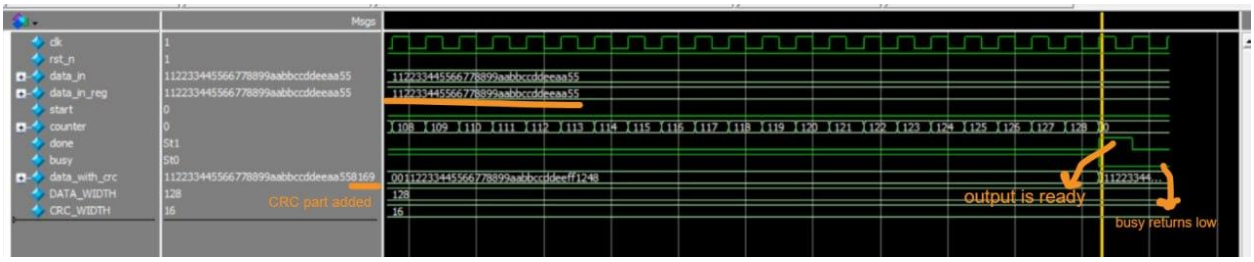
- First Test Case: Calculation terminates when *counter* = 128 ,the *done* signal gets high, the *busy* signal gets low indicating readiness for another CRC calculation and the *data_with_crc* output result is ready to be read.



- Second Test Case:



- Third Test Case:



End-to-End Data Path and Integration

The integrated system connects the three implemented processing functions through a top-level control layer. The central integration problem is not the individual algorithms but the movement of data between modules with different granularities: AES processes 128-bit blocks, CRC produces a 16-bit checksum, and UART transmits individual serial frames containing one byte at a time.

8.1 Recommended Transmission Sequence:

- Load the application plaintext and AES master key.
- Start AES encryption.
- Wait until AES asserts done and ciphertext is valid.
- Provide the protected data field to the CRC engine.
- Start the CRC calculation.
- Wait until CRC asserts done and crc_out is valid.
- Build the transmission sequence from ciphertext bytes and CRC bytes.
- Transmit each byte through UART only when UART busy is low.
- Wait for UART done before advancing to the next byte.
- After the final byte, return the integrated controller to its idle state.

8.2 Integrated State Machine:

State	Action	Exit Condition
IDLE	Wait for system start/request	Start asserted
AES_START	Launch AES	AES busy
AES_WAIT	Wait for ciphertext	AES done
CRC_START	Launch CRC	CRC busy
CRC_WAIT	Wait for checksum	CRC done
UART_LOAD	Select next byte	UART not busy
UART_SEND	Request transmission	UART busy
UART_WAIT	Wait for byte completion	UART done
COMPLETE	Report complete frame	Return to IDLE

Control and Handshake Strategy

A clean handshake protocol is essential because each processing block requires multiple clock cycles. The top-level controller should never assume that an output is available merely because an operation was started.

Signal Pattern	Meaning	Integration Rule
start	One operation requested	Assert only when the target block is ready/idle
busy = 1	Block is processing	Do not overwrite its active input/result registers
done = 1	Operation completed	Capture/use the result and advance the controller
valid	Output data is valid	Consume only when asserted
reset	Return to known state	Clear active operations and status flags

9.1 Busy Protection:

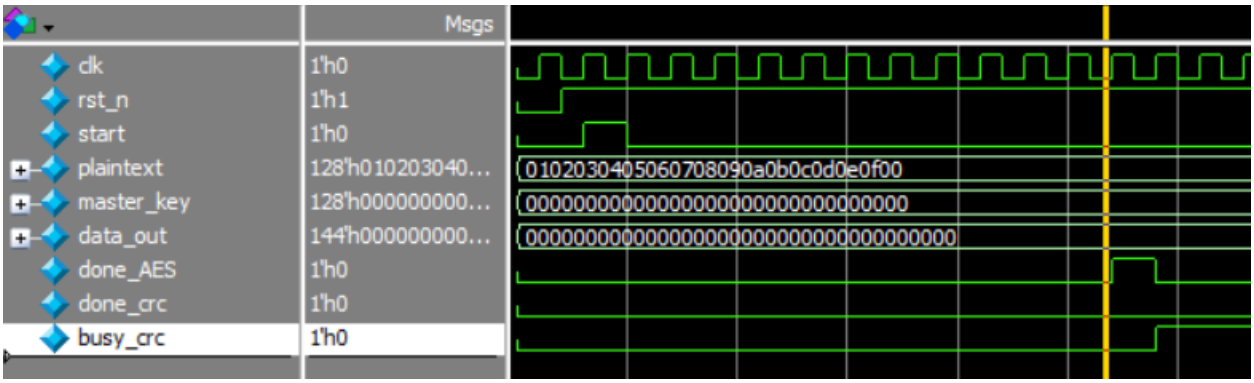
The busy signal is especially important for UART transmission. A byte must not be loaded into the UART transmitter while a previous byte is still being serialized. The same principle applies to AES and CRC: their input registers must not be changed during an active operation unless the module explicitly supports pipelining.

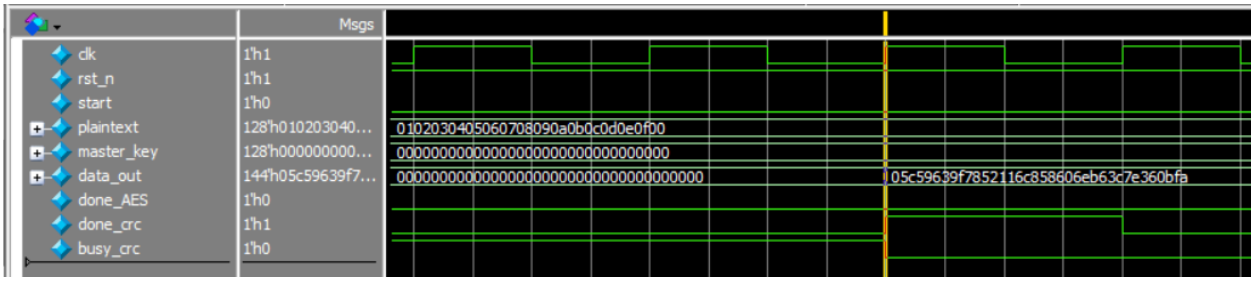
9.2 Done Pulse Handling:

If done is implemented as a one-clock pulse, the controller should sample it synchronously and transition immediately to the next state. If done remains asserted until acknowledged, the controller must include an acknowledgment or state transition that prevents repeated consumption of the same result.

AES Integration with CRC

In order to integrate the two blocks together in a wrapper, we need to use a signal which is considered as a handshake signal so when the encryption is already done, cyclic redundancy check is ready for its process.






```
# ================== AES ROUND 0 ==================
# State After Round 0  : 0102030405060708090a0b0c0d0e0f00
# Master Key           : 00000000000000000000000000000000
#
# ------------------ AES ROUND 1 ------------------
# SubBytes Out         : 7c777bf26b6fc53001672bfed7ab7663
# ShiftRows Out        : 7c6f2b636b6776f201ab7b30d777c5fe
# MixColumns Out       : 12f8a819f7dc69ca05214782b72113le
# Generated Round Key  : 626363636263636362636363626363363
# State Out (Next)     : 709bcb7a95bf0aa9674224e1d542707d
#
# ------------------ AES ROUND 2 ------------------
# SubBytes Out         : 51141fda2a0867d3852c36f8032c51ff
# ShiftRows Out        : 510836ff2a2c51da852c1fd3031467f8
# MixColumns Out       : 2adae686ad265a5c001c354cb2095566
# Generated Round Key  : 9b98c998f9fbaafb9b98c998f9fbaafb
# State Out (Next)     : b1422f1e54ddf0a79b84fcd44bf2ff9d
#
# ------------------ AES ROUND 3 ------------------
# SubBytes Out         : c82c157220c18c5c145fb048b389165e
# ShiftRows Out        : c8c1b05e205f16721489155cb32c8c48
# MixColumns Out       : 34b57e18ba9fa19f7ce21a50528c8005
# Generated Round Key  : 20bdbebdbb25772542deddded9461446
# State Out (Next)     : 1408c0a501bad6ba3e3cc78e8bca9443
```

```
# ------------------- AES ROUND 5 -------------------
# SubBytes Out          : 6e29e88a532dec3555401ea39f99a755
# ShiftRows Out         : 6e2d1e555340a78a5599e8359f29eca3
# MixColumns Out        : a370cb10584ceec40b1b8584a784c41e
# Generated Round Key   : bc9165913a566c5607b412b47888b188
# State Out (Next)      : 1fe1ae81621a82920caf9730df0c7596
#
# ------------------- AES ROUND 6 -------------------
# SubBytes Out          : c0f8e40caaa2134ffe7988049efe9d90
# ShiftRows Out         : c0a28890aa799d0cfefee44f9ef81304
# MixColumns Out        : 1ca6da1a860c77bf557ba92c45af5bc0
# Generated Round Key   : 1509d40912bdc6bd28ebaaeb947acf7a
# State Out (Next)      : 09af0e1394b1b1027d9003c7d1d594ba
#
# ------------------- AES ROUND 7 -------------------
# SubBytes Out          : 0179ab7d22c8c877ff607bc63e0322f4
# ShiftRows Out         : 01c87bf42260227dff03ab773e79c8c6
# MixColumns Out        : 0740b7b6f9bbde81c0c0f4d4bef8939c
# Generated Round Key   : 9c7b967bb4903c90a62dfa2db3242e24
# State Out (Next)      : 9b3b21cd4d2be21166ed0ef90ddcbdb8
#
# ------------------- AES ROUND 8 -------------------
# SubBytes Out          : 14e2fdbde3f198823355ab99d7867a6c
# ShiftRows Out         : 14f1ab6ce3557abd3386fd82d7e29899
# MixColumns Out        : 023ddbc65355592e3dc1b680bc58af7f
# Generated Round Key   : a305f00505280a28b1b836b82dc3a0c3
# State Out (Next)      : a1382bc3567d53068c798038919b0fbc

------------------- AES ROUND 9 -------------------
SubBytes Out          : 3207f12eb1ffed6f64b6cd0781147665
ShiftRows Out         : 32ffcd65b1b6762e6414f16f8107ed07
MixColumns Out        : 1bcc4bf9e7b2c1cb1acea69c7c4ea4fa
Generated Round Key   : baa8aaa80b109c100e389638ad3d663d
State Out (Next)      : a164e151eca25ddb14f630a4d173c2c7

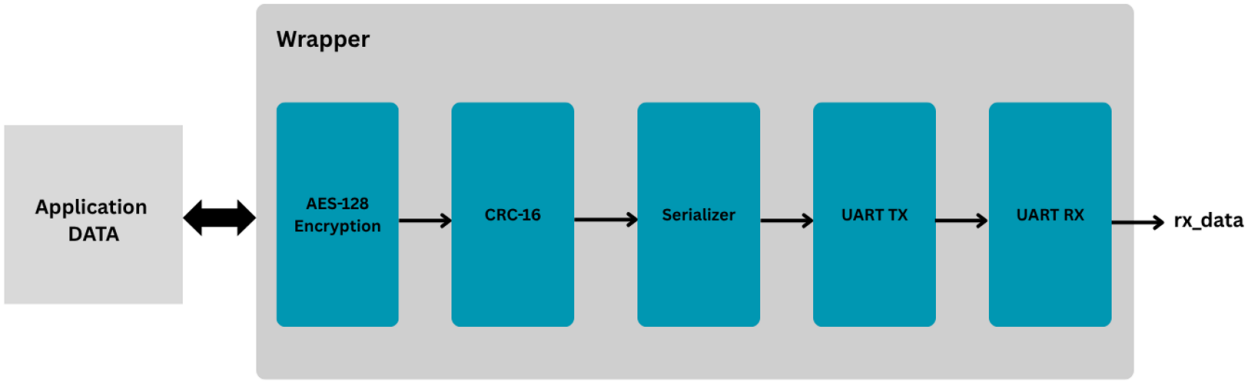
=================== FINAL ROUND 10 ===================
SubBytes Out          : 3243f8d1ce3a4cb9fa4204493e8f25c6
ShiftRows Out         : 323a04c6ce4225d1fa8ff8b93e434c49
Final Round Key       : 37ff92ff39c704c732d798d7887f327f
Final Ciphertext      : 05c59639f7852116c858606eb63c7e36
=======================================================

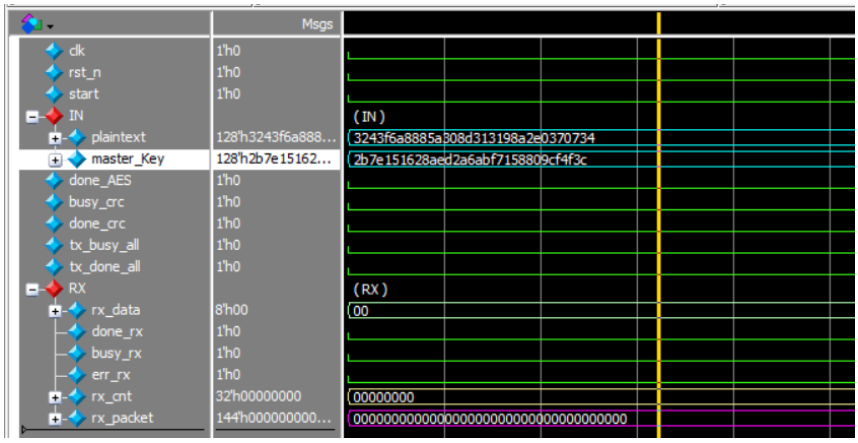

[SUCCESS] Final Output (Ciphertext + CRC16) : 05c59639f7852116c858606eb63c7e360bfa
** Note: $finish    : ./tb_TOP_LEVEL.sv(65)
   Time: 2970 ns  Iteration: 0  Instance: /tb_TOP_LEVEL
```

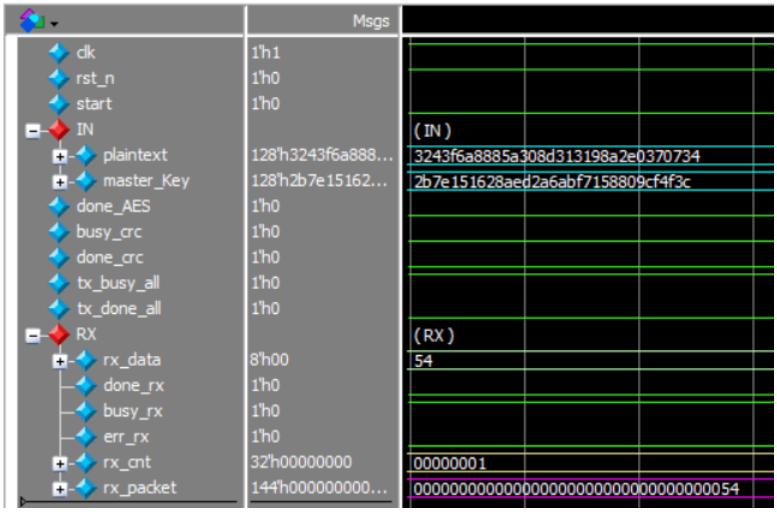
System Integration

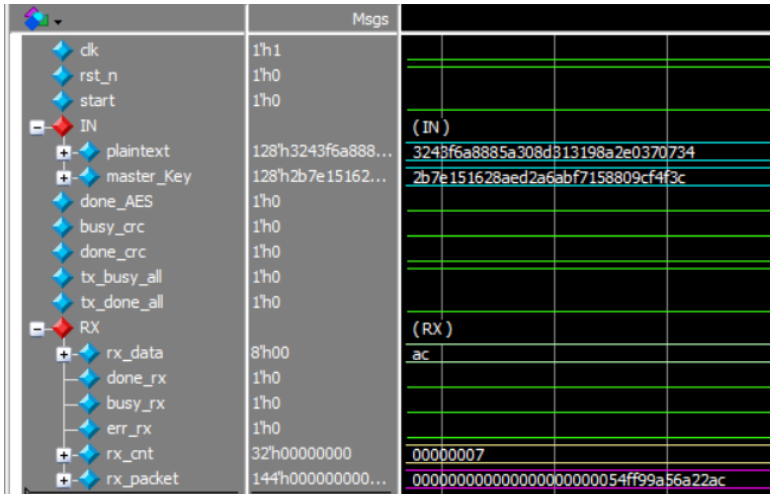
The whole system is integrated and combined in one wrapper so, data is encrypted using AES block, after that Checked using CRC block, finally is transmitted through the UART Communication Protocol.

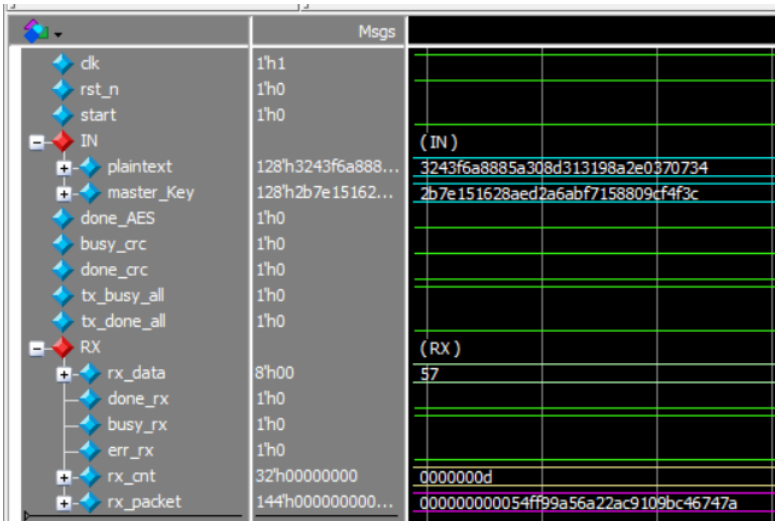
In order to transmit the data from the CRC to UART there is a Serializer block that has been designed so that the Transmitter could capture the data easily.

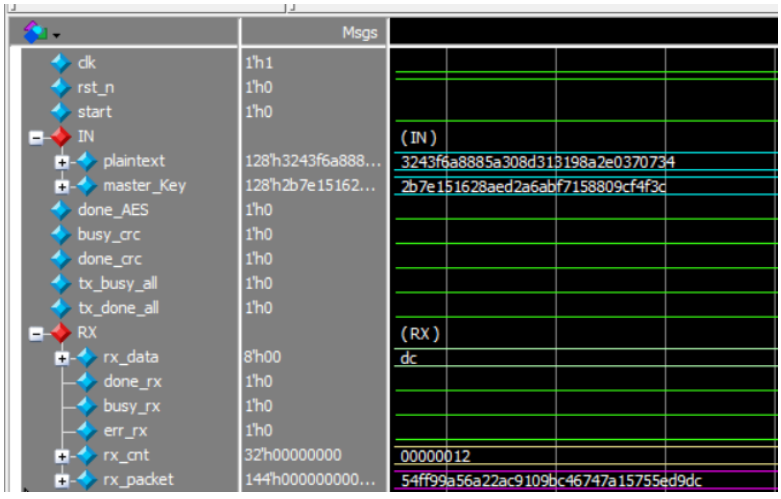







Msgs
clk
rst_n
start
IN
plaintext
master_Key
done_AES
busy_crc
done_crc
tx_busy_all
tx_done_all
RX
rx_data
done_rx
busy_rx
err_rx
rx_cnt
rx_packet
1'h1
1'h0
1'h0
128'h3243f6a888...
128'h2b7e15162...
32'h00000000
144'h000000000...
(IN)
3243f6a8885a308d313198a2e0370734
2b7e151628aed2a6abf7158809cf4f3c
(RX)
ac
00000007
000000000000000000000054ff99a56a22ac


Msgs
clk
rst_n
start
IN
plaintext
master_Key
done_AES
busy_crc
done_crc
tx_busy_all
tx_done_all
RX
rx_data
done_rx
busy_rx
err_rx
rx_cnt
rx_packet
1'h1
1'h0
1'h0
128'h3243f6a888...
128'h2b7e15162...
32'h00000000
144'h000000000...
(IN)
3243f6a8885a308d313198a2e0370734
2b7e151628aed2a6abf7158809cf4f3c
(RX)
57
0000000d
000000000054ff99a56a22ac9109bc46747a


Msgs
clk
rst_n
start
IN
plaintext
master_Key
done_AES
busy_crc
done_crc
tx_busy_all
tx_done_all
RX
rx_data
done_rx
busy_rx
err_rx
rx_cnt
rx_packet
1'h1
1'h0
1'h0
128'h3243f6a888...
128'h2b7e15162...
32'h00000000
144'h000000000...
(IN)
3243f6a8885a308d313198a2e0370734
2b7e151628aed2a6abf7158809cf4f3c
(RX)
dc
00000012
54ff99a56a22ac9109bc46747a15755ed9dc

```
# [450 ns] STAGE 1 - AES done
#          ciphertext      = 54ff99a56a22ac9109bc46747a15755e
#
# [3050 ns] STAGE 2 - CRC done
#          CRC-16          = d9dc
#          data_with_crc   = 54ff99a56a22ac9109bc46747a15755ed9dc  (18 bytes: 16 data + 2 CRC)
# [3090 ns] STAGE 3 - TX sends byte = 54
# [1093850 ns] STAGE 4 - RX byte  0 = 54
# [1145930 ns] STAGE 3 - TX sends byte = ff
# [2239610 ns] STAGE 4 - RX byte  1 = ff
# [2291690 ns] STAGE 3 - TX sends byte = 99
# [3385370 ns] STAGE 4 - RX byte  2 = 99
# [3437450 ns] STAGE 3 - TX sends byte = a5
# [4531130 ns] STAGE 4 - RX byte  3 = a5
# [4583210 ns] STAGE 3 - TX sends byte = 6a
# [5676890 ns] STAGE 4 - RX byte  4 = 6a
# [5728970 ns] STAGE 3 - TX sends byte = 22
# [6822650 ns] STAGE 4 - RX byte  5 = 22
# [6874730 ns] STAGE 3 - TX sends byte = ac
# [7968410 ns] STAGE 4 - RX byte  6 = ac
# [8020490 ns] STAGE 3 - TX sends byte = 91
# [9114170 ns] STAGE 4 - RX byte  7 = 91
# [9166250 ns] STAGE 3 - TX sends byte = 09
# [10259930 ns] STAGE 4 - RX byte  8 = 09
# [10312010 ns] STAGE 3 - TX sends byte = bc
# [11405690 ns] STAGE 4 - RX byte  9 = bc
# [11457770 ns] STAGE 3 - TX sends byte = 46
# [12551450 ns] STAGE 4 - RX byte 10 = 46
# [12603530 ns] STAGE 3 - TX sends byte = 74
# [13697210 ns] STAGE 4 - RX byte 11 = 74
# [13749290 ns] STAGE 3 - TX sends byte = 7a
# [14842970 ns] STAGE 4 - RX byte 12 = 7a
# [14895050 ns] STAGE 3 - TX sends byte = 15
# [15988730 ns] STAGE 4 - RX byte 13 = 15
# [16040810 ns] STAGE 3 - TX sends byte = 75
# [17134490 ns] STAGE 4 - RX byte 14 = 75
# [17186570 ns] STAGE 3 - TX sends byte = 5e
# [18280250 ns] STAGE 4 - RX byte 15 = 5e
# [18332330 ns] STAGE 3 - TX sends byte = d9
# [19426010 ns] STAGE 4 - RX byte 16 = d9
# [19478090 ns] STAGE 3 - TX sends byte = dc
# [20571770 ns] STAGE 4 - RX byte 17 = dc
#
# [20623830 ns] STAGE 3 - serializer done (all 18 bytes sent)
```

```
# ==================== FINAL RECEIVER OUTPUT ====================
#  bytes received : 18 (expected 18)
#  ciphertext     : 54ff99a56a22ac9109bc46747a15755e
#  CRC-16         : d9dc
#  full packet    : 54ff99a56a22ac9109bc46747a15755ed9dc
```

Timing and Synthesizability Considerations

The system is clocked RTL and is intended to be synthesizable. Because AES and CRC contain iterative logic and UART contains baud-rate timing, the main timing considerations are combinational depth, register placement, and the number of clock cycles allocated to each operation.

10.1 AES Timing:

- An iterative AES architecture reduces hardware area by reusing the round datapath.
- The critical path can be influenced by the S-box implementation, MixColumns logic, and key-expansion logic.
- Registering intermediate results between rounds can improve timing at the cost of additional latency.
- A fully combinational 10-round implementation may reduce cycle latency but typically increases area and timing pressure.

10.2 CRC Timing:

- A bit-serial CRC implementation uses very little hardware but requires multiple cycles per byte.
- A byte-parallel implementation can process one byte per cycle but uses more combinational logic.
- The selected implementation is matched to the required throughput of the UART link.

10.3 UART Timing:

- UART baud timing must be derived consistently from the system clock.
- The transmitter must maintain the configured bit period with sufficient accuracy.
- The receiver sample near the center of each bit period for robust operation.

10.4 Reset Strategy:

Reset behavior initialize all state machines, counters, shift registers, busy flags, done flags, and data-valid indicators. After reset, the integrated system returns to a defined idle state without starting an unintended encryption, CRC calculation, or UART transmission.

Design Decisions and Engineering Rationale

Decision	Rationale
Modular RTL	Allows AES, CRC, and UART to be verified independently and reused in other systems.
AES-128	Provides a standardized 128-bit block cipher with a compact key size suitable for an RTL learning/project implementation.
CRC-16/XMODEM	Provides a standardized 16-bit checksum with well-defined parameters and a known validation vector.
Handshake-based control	Prevents data overwrite and makes multi-cycle processing safe.
Byte-oriented UART interface	Matches the natural granularity of UART while allowing the top level to serialize AES/CRC results.
Iterative processing	Offers a practical balance between hardware resource usage and latency for a project-scale RTL implementation.

11.1 Security and Integrity:

AES and CRC serve different purposes and should not be treated as interchangeable. AES provides confidentiality when used with a properly managed secret key, while CRC detects accidental transmission errors. CRC is not a cryptographic authentication mechanism and should not be used to establish authenticity or protect against an intentional attacker modifying the message.

11.2 CRC After Encryption:

A communication link can corrupt ciphertext bits just as it can corrupt plaintext bits. Computing CRC over the defined transmitted protected data allows the receiver to detect many accidental bit errors before accepting the data as valid.

Limitations and Future Improvements

- Add a complete AES decryption path for bidirectional secure communication.
- Add a formal frame format with synchronization, length, sequence number, and explicit CRC coverage.
- Add CRC error reporting at the receiver and a defined retransmission policy.
- Optimize the S-box implementation for the selected FPGA/ASIC technology.
- Add assertions and/or formal verification for key control properties such as busy exclusivity and state transitions.
- Add randomized constrained verification and functional coverage.
- Perform post-synthesis timing analysis and document area, power, and maximum-frequency results.

Final Specifications

Parameter	Selected Target
AES	AES-128, 128-bit key, 128-bit block, 10 rounds
AES mode	Direct block encryption (no chaining mode)
CRC	CRC-16/XMODEM, polynomial 0x1021, initial value 0x0000, RefIn/RefOut false, XorOut 0x0000
Protected payload	18 bytes (16-byte ciphertext + 2-byte CRC)
UART	Single lane, start/data/stop framing, configurable baud rate
Control style	Synchronous start/busy/done handshaking between all stages
System status	Individual modules implemented and integrated into a complete RTL system

Conclusion

The completed project demonstrates the integration of three important digital hardware functions into a single RTL communication system. UART provides the physical serial data transport, AES-128 provides encryption, and CRC-16/XMODEM provides error detection. The resulting architecture illustrates how independently verified RTL blocks can be coordinated through control states and handshaking to form a practical hardware subsystem.

References

[1] Basu, D., Kole, D. K., Rahaman, H.: “Implementation of AES algorithm in UART module for secured data transfer.” International Conference on Advances in Computing and Communications (2012), pp. 142–145.

[2] Katkade, P., Phade, G.: “Application of AES algorithm for data security in serial communication.” International Conference on Inventive Computation Technologies (2016), pp. 1–5.

[3] Kala, J., Panda, J., Tanwar, L.: “FPGA implementation of a high throughput Low Power Advanced Encryption Standard (AES-128) Cipher.” 10th International Conference on Signal Processing and Integrated Networks (SPIN) (2023), pp. 367–372.

[4] Kinshuk et al.: “Promoting Sustainable Security by Implementing AES Algorithm for Enhanced Confidentiality in UART Serial Communication.” Springer Nature Link.

[5] National Institute of Standards and Technology (NIST): “Advanced Encryption Standard (AES), FIPS PUB 197.” U.S. Department of Commerce.

[6] CRC-16/XMODEM parameterization: polynomial 0x1021, init 0x0000, RefIn/RefOut false, XorOut 0x0000, check value 0x31C3, as catalogued in standard CRC parameter registries.